Z

ZENGFY ENGINEERING STANDARDS

Zengfy Sovereign Registration & Identity Engineering Masterclass

এন্টারপ্রাইজ গ্রেড নিরাপদ, ব্যয়-সাশ্রয়ী (Zero-Cost Verification) এবং সার্বভৌম রেজিস্ট্রেশন সিস্টেম নির্মাণের সম্পূর্ণ হ্যান্ডবুক

📂 পাথ: core/docs/Registration

🛡️ রুল: Day 0 Minimalist Flow (৩টি ফিল্ড)

⚖️ সাইবার নিরাপত্তা আইন ২০২৩ কমপ্লায়েন্ট

📅 সংস্করণ: অক্টোবর ২০২৬

মডিউল 00 / ১১

Zengfy Sovereign Registration & Identity Engineering Masterclass

> ***ডকুমেন্টেশন রুট পাথ:*** `P:\Zengfy-Ecosystem\core\docs\Registration`
> ***উদ্দেশ্য:*** *এন্টারপ্রাইজ সফটওয়্যার ইঞ্জিনিয়ারিং স্ট্যান্ডার্ডে নিরাপদ, ব্যয়-সাশ্রয়ী এবং স্কেলেবল ইউজার রেজিস্ট্রেশন সিস্টেম নির্মাণ।*
> ***ইকোসিস্টেম সংস্করণ:*** *Zengfy Core v1.0 (Day 0 Minimalist Flow)*
> ***সর্বশেষ হালনাগাদ:*** *অক্টোবর ২০২৬*

---

১. নির্বাহী সারসংক্ষেপ (Executive Summary)

একটি ডিজিটাল প্ল্যাটফর্ম বা ইকোসিস্টেমের প্রথম এবং প্রধান প্রবেশদ্বার হলো **রেজিস্ট্রেশন এবং আইডেন্টিটি সিস্টেম (Registration & Identity System)**। অধিকাংশ প্রজেক্টে রেজিস্ট্রেশনকে শুধুমাত্র একটি সাধারণ ওয়েব ফর্ম ও ডেটাবেজ ইনসার্ট অপারেশন হিসেবে ভাবা হয়। কিন্তু বাস্তব প্রডাকশন পরিবেশে এটি সিকিউরিটি, ক্রিপ্টোগ্রাফি, সিস্টেম রেজিলিয়েন্স, আইনি কমপ্লায়েন্স এবং ইউজার এক্সপেরিয়েন্সের এক জটিল সমন্বয়।

Zengfy ইকোসিস্টেমের জন্য আমাদের কৌশলগত সিদ্ধান্ত:

1. **Day 0 (প্রথম দিন):** অতি-সহজ ও ঘর্ষণহীন (Frictionless) রেজিস্ট্রেশন। ইউজার কেবল **৩টি ফিল্ড** পূরণ করবেন:
   - **পুরো নাম (Full Name)**
   - **ইমেইল ঠিকানা (Email Address)**
   - **পাসওয়ার্ড (Password)**
     (অথবা Google 1-Tap Sign-In এর মাধ্যমে ০-ফিল্ডে তাৎক্ষণিক একাউন্ট খোলা)।
2. **জিরো-কস্ট ভেরিফিকেশন (Zero-Dollar Verification):** প্রাথমিক পর্যায়ে ব্যয়বহুল SMS OTP সম্পূর্ণ পরিহার করে নিরাপদ ক্রিপ্টোগ্রাফিক ইমেইল ভেরিফিকেশন লিংক চালু রাখা।
3. **প্রগ্রেসিভ প্রোফাইলিং ও ব্যাজ সিস্টেম (Progressive Profiling & Badges):** ফোন নাম্বার, জাতীয় পরিচয়পত্র (NID) বা উন্নত নাগরিক ভেরিফিকেশন পরবর্তী সময়ে একাউন্ট সেন্টারে (Account Center) ব্যাজ বা ট্রাস্ট টিয়ার (Trust Tier) আকারে যুক্ত হবে।

---

২. মাস্টারক্লাস ডকুমেন্টেশনের সূচিপত্র (Curriculum Index)

এই মাস্টারক্লাসটি মোট ১০টি গভীর মডিউল ও পূর্ণাঙ্গ কোডবেসের সমন্বয়ে গঠিত:

ক্রম	ডকুমেন্টের নাম	মূল বিষয়বস্তু
01	`01-Registration-Fundamentals-And-Theory.md`	আইডেন্টিটি থিওরি, ৩-ফিল্ড সাইন-আপ ফিলোসফি, কনভার্সন অপটিমাইজেশন ও প্রগ্রেসিভ প্রোফাইলিং।
02	`02-System-Design-And-Architecture.md`	হাই-লেভেল ও লো-লেভেল আর্কিটেকচার, ডেটাবেজ স্কিমা (PostgreSQL), স্টেট মেশিন ও ইভেন্ট আর্কিটেকচার।
03	`03-Security-Cryptography-And-Hardening.md`	Argon2id হ্যাশিং, সল্ট ও পেপার, কনস্ট্যান্ট-টাইম চেক, OWASP ASVS গাইডলাইন ও রেট লিমিটিং।
04	`04-Verification-Mechanisms-Email-Vs-Phone.md`	ইমেইল বনাম SMS এর আর্থিক ও প্রযুক্তিগত তুলনা (টেলকো খরচ ৳০.৩০-০.৫০), SPF/DKIM/DMARC ও টোকেন মেকানিজম।
05	`05-Real-World-Edge-Cases-And-Production-Bugs.md`	রেস কন্ডিশন, ইউনিকোড হোমোগ্রাফ আক্রমণ, প্লাস-অ্যাড্রেসিং, ডিসপোজেবল ইমেইল ফিল্টার ও নেটওয়ার্ক ফেইলিউর।
06	`06-Case-Studies-Industry-Analysis.md`	গুগল (Glif Flow), গিটহাব, স্ট্রাইপ এবং দেশীয় বিকাশ/নগদ/পাঠাও-এর বাস্তব কেস স্টাডি ও শিক্ষা।
07	`07-Zengfy-Implementation-Blueprint.md`	Zengfy প্ল্যাটফর্মের স্পেসিফিক ব্লুপ্রিন্ট, সার্বভৌম ট্রাস্ট টিয়ার (Tier 0 থেকে Tier 3) ও মাইক্রোসার্ভিস রোডম্যাপ।
08	`08-IT-Company-Roles-And-Engineering-Workflow.md`	একটি স্বনামধন্য আইটি কোম্পানির সকল রোলের দায়িত্ব (PM, UX, CISO, Legal, Backend, QA, DevOps) ও সাইবার আইন ২০২৩।
09	`09-Day-0-Root-Scenario-Implementation-Guide.md`	ডে ০-এর পূর্ণাঙ্গ ব্যাকএন্ড কন্ট্রোলার, সার্ভিস, সিকিউর টোকেন জেনারেটর এবং টেলউইন্ড রেসপন্সিভ ইউজার ইন্টারফেস।
10	`10-Testing-QA-And-DevOps-Runbook.md`	অটোমেটেড ইউনিট ও ইন্টিগ্রেশন টেস্ট, পেন-টেস্টিং প্ল্যান, Docker Compose এনভায়রনমেন্ট এবং ইনসিডেন্ট রানবুক।

---

৩. প্রযুক্তি স্ট্যাক (Technology Stack)

- **ব্যাকএন্ড ফ্রেমওয়ার্ক:** Node.js (TypeScript) / PHP (Laravel 11) এন্টারপ্রাইজ প্যাটার্ন।
- **ডেটাবেজ:** PostgreSQL 16 (UUIDv7, CITEXT, Partial Indexes, Row Level Security)।
- **ক্যাশ ও রেট-লিমিটিং:** Redis 7 (Leaky Bucket / Sliding Window Counter)।
- **ক্রিপ্টোগ্রাফি:** Argon2id (Memory: 64MB, Iterations: 3, Parallelism: 4) ও HMAC-SHA256।
- **ফ্রন্টএন্ড:** Tailwind CSS v3/v4, হ্যান্ডক্রাফটেড রেসপন্সিভ মোবাইল ইউআই, ফ্লেক্সিবল জাভাস্ক্রিপ্ট / ব্লেড কম্পোনেন্ট।
- **লোকাল মেল সার্ভার:** Mailhog (লোকালে কোনো আসল ইমেইল না পাঠিয়ে ব্রাউজারে ইনবক্স ভিউ করার জন্য)।

---

৪. কীভাবে শুরু করবেন? (Quick Start)

আপনি যদি সম্পূর্ণ কনসেপ্ট ও বাস্তব কোড একসাথে শিখতে চান:

1. প্রথমে `01` **থেকে** `07` **নম্বর মডিউল** পড়ে আর্কিটেকচার, সিকিউরিটি এবং বাস্তব সমস্যার সমাধানগুলো হৃদয়াঙ্গম করুন।
2. এরপর `08` **নম্বর মডিউল** দেখে একটি পূর্ণাঙ্গ প্রফেশনাল সফটওয়্যার টিমের ভূমিকা এবং বাংলাদেশের সাইবার নিরাপত্তা আইন ২০২৩ ও ডেটা প্রোটেকশন বিধিমালার গাইডলাইন দেখুন।
3. সরাসরি কোড প্র্যাকটিসের জন্য `09` **নম্বর মডিউল** থেকে ব্যাকএন্ড সার্ভিস ও ফ্রন্টএন্ড ফর্মটি আপনার প্রজেক্টে যুক্ত করুন।
4. `10` **নম্বর মডিউল** অনুযায়ী টেস্ট রান করুন এবং ডকার এনভায়রনমেন্টে চালিয়ে দেখুন।

মডিউল ০১: রেজিস্ট্রেশন ফান্ডামেন্টালস ও কোর থিওরি (Fundamentals & Core Theory)

📖 *মূল সূচিপত্র (README)* | *পরবর্তী মডিউল* ➡️

> ***লক্ষ্য:*** *আইডেন্টিটি, প্রমাণীকরণ ও অনুমোদনের তাত্ত্বিক পার্থক্য স্পষ্ট করা এবং কেন শুরুতে কেবল ৩টি ফিল্ড রাখা উচিত তা মনস্তাত্ত্বিক ও প্রযুক্তিগতভাবে বোঝা।*

---

১. আইডেন্টিটি, প্রমাণীকরণ ও অনুমোদন (The Holy Trinity of Auth)

সফটওয়্যার ইঞ্জিনিয়ারিংয়ে প্রায়ই তিনটি ধারণাকে গুলিয়ে ফেলা হয়:

```
[ ইউজার আইডেন্টিটি (Identity) ]  ⟶ "আপনি কে?" (নাম, ইমেইল, হ্যান্ডেল)
                │
                ▼
[ প্রমাণীকরণ (Authentication) ]  ⟶ "আপনি কি আসলেই সেই ব্যক্তি?" (পাসওয়ার্ড, ওটিপি, পাসকি)
                │
                ▼
[ অনুমোদন (Authorization) ]      ⟶ "আপনার কি এই সিস্টেমে প্রবেশের অধিকার আছে?" (রোল, পারমিশন)
```

1. **আইডেন্টিটি (Identity):** সিস্টেমে ইউজারের একক অস্তিত্ব। এটি একটি অনন্য আইডেন্টিফায়ার (যেমন: `sazzad@zengfy.com` বা UUID)।
2. **অথেনটিকেশন বা প্রমাণীকরণ (Authentication - AuthN):** প্রেরিত আইডেন্টিটি সত্য কি না তা যাচাই করার প্রক্রিয়া (যেমন: সঠিক পাসওয়ার্ড প্রদান করা)।
3. **অথরাইজেশন বা অনুমোদন (Authorization - AuthZ):** যাচাইকৃত আইডেন্টিটিকে কোন কোন কাজ করার অনুমতি দেওয়া হয়েছে তা নির্ধারণ করা (যেমন: ইউজার কি সাধারণ নাগরিক, নাকি অ্যাডমিন?)।

---

২. ডে ০ ফিলোসফি: ৩টি ফিল্ড কেন? (The Day 0 Three-Field Philosophy)

বাস্তব প্রজেক্টগুলোতে অনেক সময় শুরুতেই ইউজারের কাছ থেকে অজস্র তথ্য চেয়ে বসা হয়—ফোন নাম্বার, ঠিকানা, বয়স, লিঙ্গ, এনআইডি নাম্বার ইত্যাদি। এটি প্রোডাক্ট ডেভেলপমেন্টের অন্যতম বড় ভুল।

২.১ কনভার্সন ড্রপ-অফ ম্যাথ (Conversion Drop-off Law)

ই-কমার্স ও সাস (SaaS) ইন্ডাস্ট্রির বহুল পরিচিত গবেষণা অনুযায়ী:

- রেজিস্ট্রেশন ফর্মে প্রতি ১টি অতিরিক্ত ইনপুট ফিল্ড যুক্ত করার ফলে **১০% থেকে ১৫% ভিজিটর ফর্ম পূরণ না করেই পেজ বন্ধ করে চলে যায়**।
- ফোন নাম্বারে বাধ্যতামূলক SMS OTP যুক্ত করলে নেটওয়ার্ক ডিলে এবং ইউজারের দ্বিধার কারণে সাইন-আপ ড্রপ-অফ আরও **২৫% থেকে ৩৫% বৃদ্ধি পায়**।

```
ফর্ম ফিল্ডের সংখ্যা        ইউজার কমপ্লিশন রেট (Conversion Rate)
──────────────────────      ──────────────────────────────────────
৩টি ফিল্ড (Name/Email/Pass)  ████████████████████ 85% - 92%
৬টি ফিল্ড (+Phone, DOB, Gen) ███████████ 45% - 52%
১০+ ফিল্ড (+NID, Address)    ████ 15% - 22%  (মারাত্মক বিপর্যয়!)
```

২.২ ডে ০ রেজিস্ট্রেশনের গোল্ডেন ফিল্ড ত্রয়ী

Zengfy প্ল্যাটফর্মে প্রথম দিনে আমরা শুধু তিনটি মৌলিক ফিল্ড নিচ্ছি:

1. **Full Name (সম্পূর্ণ নাম):** ইউজারের মানবীয় পরিচিতি ও কমিউনিকেশন অ্যাড্রেসের জন্য।
2. **Email Address (ইমেইল ঠিকানা):** গ্লোবাল ইউনিক আইডেন্টিফায়ার, পাসওয়ার্ড রিকভারি চ্যানেল এবং ফ্রিতে ভেরিফিকেশন পাঠানোর মাধ্যম।
3. **Password (পাসওয়ার্ড):** প্রাথমিক প্রমাণীকরণ মেকানিজম (কমপক্ষে ৮ অক্ষর, মিশ্র ক্যারেক্টার)।

> ⚡ **TIP (টিপস)**
>
> গুগল ওয়ান-ট্যাপ (Google 1-Tap) সাইন-ইন চালু থাকলে এই ৩টি ফিল্ডও ইউজারের টাইপ করতে হয় না; সিঙ্গেল ক্লিকে ০-ফিল্ডে একাউন্ট তৈরি হয়ে যায়।

---

৩. প্রগ্রেসিভ প্রোফাইলিং (Progressive Profiling & Trust Tiers)

ইউজারের সব তথ্য প্রথম দিনে না নিয়ে কখন নেওয়া হবে? সমাধান হলো **প্রগ্রেসিভ প্রোফাইলিং**।

ইউজার যখন সিস্টেমে তার অ্যাক্টিভিটি বাড়াবেন এবং উন্নত ফিচারের সুবিধা নিতে চাইবেন, তখন ধাপে ধাপে অতিরিক্ত তথ্য চাওয়া হবে:

1. **টিয়ার ১ (বেসিক মেম্বার):** ৩-ফিল্ড রেজিস্ট্রেশন সম্পন্ন। ইউজার কনটেন্ট পড়তে পারবেন, পাবলিক পোস্ট দেখতে পারবেন।
2. **টিয়ার ২ (ভেরিফাইড ইমেইল মেম্বার):** ইমেইল লিংক ক্লিক করেছেন। এখন ইউজার কমেন্ট করতে পারবেন এবং সোশ্যাল ফিডে অংশ নিতে পারবেন।
3. **টিয়ার ৩ (কমিউনিটি মেম্বার):** একাউন্ট সেন্টারে গিয়ে ফোন নাম্বার যুক্ত করেছেন। হাইপারলোকাল গ্রুপ তৈরি করতে পারবেন।
4. **টিয়ার ৪ (সার্বভৌম নাগরিক মেম্বার):** এনআইডি বা স্মার্টকার্ড ভেরিফিকেশন সম্পন্ন। অফিশিয়াল ভোট ও সিভিক ডিসিশনে অংশ নিতে পারবেন।

```
flowchart LR
    A["Day 0 Sign-Up
(Name + Email + Pass)"] ⟶ B["Tier 1: Guest/Basic
(Browse, Read)"]
    B ⟶ C["Email Click
(Link Token)"]
    C ⟶ D["Tier 2: Verified Citizen
(Post, Comment)"]
    D ⟶ E["Account Center
(Add Phone/NID Badges)"]
    E ⟶ F["Tier 3: Sovereign Civic
(Vote, Official Beacons)"]
```

এই পদ্ধতির ফলে রেজিস্ট্রেশনের দিনে কোনো ইউজার আটকে থাকেন না, প্ল্যাটফর্মের গ্রোথ দ্রুত গতিতে বাড়ে, এবং ডেটাবেজে ভুয়া ইউজারের অপ্রয়োজনীয় জটলা তৈরি হয় না।

> 📖 *মূল সূচিপত্র (README)* | *পরবর্তী মডিউল* ➡️

মডিউল 02 / ১১

মডিউল ০২: সিস্টেম ডিজাইন ও সফটওয়্যার আর্কিটেকচার (System Design & Architecture)

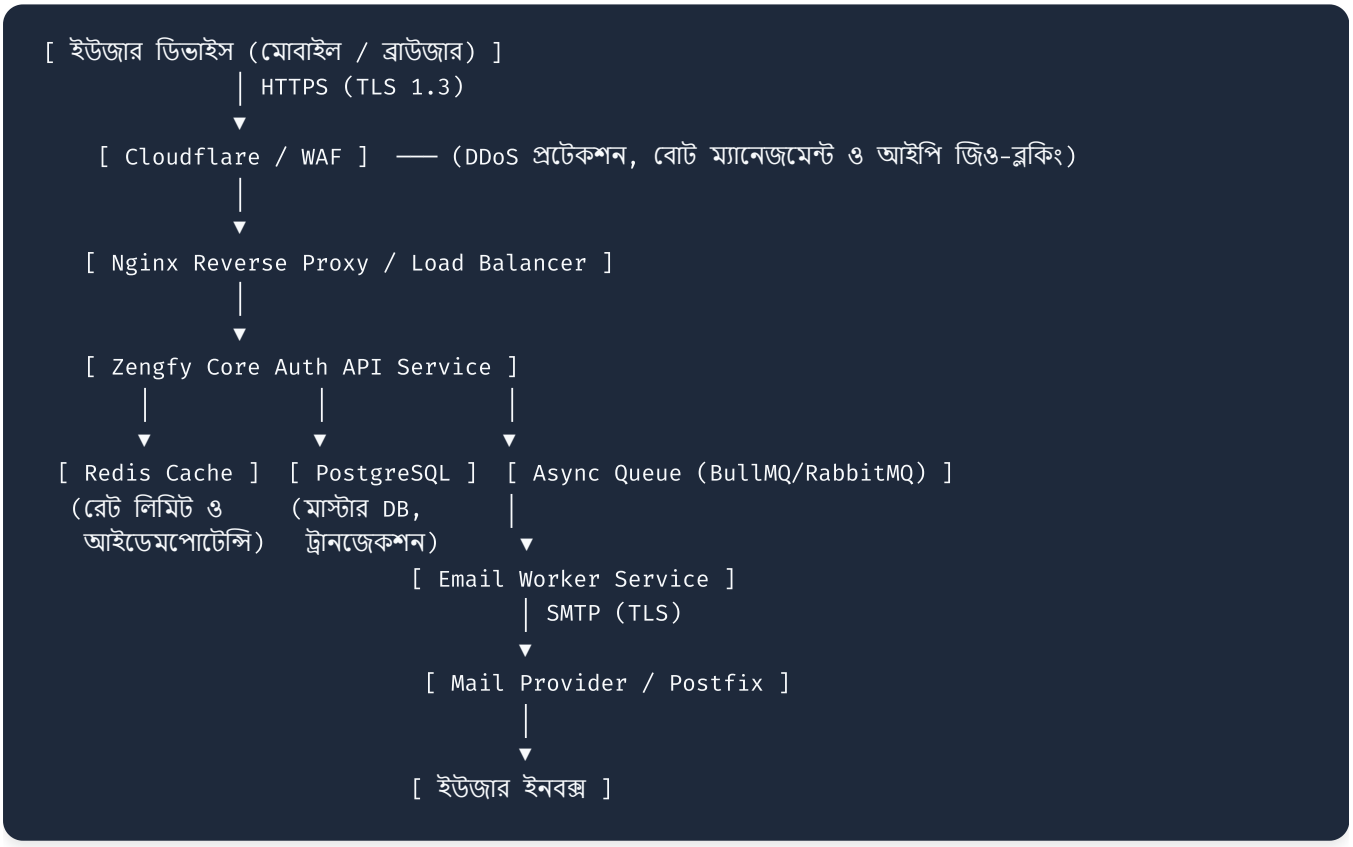
> ⬅️ *পূর্ববর্তী মডিউল* | 📖 *মূল সূচিপত্র (README)* | *পরবর্তী মডিউল* ➡️

> ***লক্ষ্য:*** *রেজিস্ট্রেশন সাবসিস্টেমের হাই-লেভেল এবং লো-লেভেল আর্কিটেকচার ডিজাইন করা, স্টেট মেশিন মডেলিং এবং রিলেশনাল ডেটাবেজের অপ্টিমাইজড স্কিমা প্রস্তুত করা।*

---

১. হাই-লেভেল সিস্টেম আর্কিটেকচার (High-Level Architecture)

একটি এন্টারপ্রাইজ গ্রেড রেজিস্ট্রেশন সিস্টেমের ব্লক আর্কিটেকচার:



কেন অ্যাসিঙ্ক্রোনাস ইমেইল কিউ (Async Email Queue) অপরিহার্য?

রেজিস্ট্রেশন রিকোয়েস্ট আসার পর ব্যাকএন্ডে কখনোই সরাসরি সিঙ্ক্রোনাসলি (Synchronously) SMTP সার্ভারের সাথে সংযোগ তৈরি করে ইমেইল পাঠানো উচিত নয়। কারণ:

- SMTP হ্যান্ডশেক এবং ইমেইল ডেলিভারিতে ১ থেকে ৩ সেকেন্ড সময় লাগতে পারে।
- কোনো কারণে SMTP প্রোভাইডার ডাউন থাকলে বা রেসপন্স করতে দেরি হলে পুরো রেজিস্ট্রেশন API টাইম-আউট হয়ে যাবে।
- সঠিক আর্কিটেকচার হলো: ডেটাবেজে ইউজার ডাটা এবং ভেরিফিকেশন টোকেন লিখে তাৎক্ষণিকভাবে কিউ-তে একটি ইভেন্ট পুশ করা (`UserRegisteredEvent`) এবং ইউজারকে সাথে সাথে `201 Created` রেসপন্স দেওয়া। ব্যাকগ্রাউন্ডে ওয়ার্কার প্রসেস সেই কিউ থেকে ইমেইল পাঠাবে।

---

২. ডেটাবেজ স্কিমা ডিজাইন (PostgreSQL Relational Schema)

আমরা ইন্ডাস্ট্রি-স্ট্যান্ডার্ড PostgreSQL 16 এর সুবিধাগুলো ব্যবহার করে অপ্টিমাইজড স্কিমা ডিজাইন করেছি:

```sql
-- ১. UUID এক্সটেনশন ও CITEXT (কেস-ইনসেনসিটিভ টেক্সট) এক্সটেনশন চালু করা
CREATE EXTENSION IF NOT EXISTS "uuid-ossp";
CREATE EXTENSION IF NOT EXISTS "citext";

-- ২. ইউজার স্ট্যাটাস এনুমারেটর (State Machine Enum)
CREATE TYPE user_account_status AS ENUM (
    'PENDING_VERIFICATION',
    'ACTIVE',
    'SUSPENDED',
    'DEACTIVATED'
);

-- ৩. কোর ইউজার টেবিল (Users Table)
CREATE TABLE users (
    id UUID PRIMARY KEY DEFAULT uuid_generate_v4(),
    full_name VARCHAR(100) NOT NULL,
    email CITEXT UNIQUE NOT NULL, -- স্বয়ংক্রিয় কেস-ইনসেনসিটিভ (যেমন: User@Domain.com == user@domain
    password_hash VARCHAR(255) NOT NULL,
    status user_account_status NOT NULL DEFAULT 'PENDING_VERIFICATION',
    email_verified_at TIMESTAMPTZ NULL,
    created_at TIMESTAMPTZ NOT NULL DEFAULT CURRENT_TIMESTAMP,
    updated_at TIMESTAMPTZ NOT NULL DEFAULT CURRENT_TIMESTAMP,
    deleted_at TIMESTAMPTZ NULL -- সফট ডিলিট সাপোর্ট
);

-- ইনডেক্সিং অপ্টিমাইজেশন
CREATE INDEX idx_users_email ON users(email);
CREATE INDEX idx_users_status ON users(status);
CREATE INDEX idx_users_created_at ON users(created_at);

-- ৪. ইমেইল ভেরিফিকেশন টোকেন টেবিল (Tokens Table)
CREATE TABLE email_verification_tokens (
    id BIGSERIAL PRIMARY KEY,
    user_id UUID NOT NULL REFERENCES users(id) ON DELETE CASCADE,
    token_hash VARCHAR(64) NOT NULL, -- প্লেইনটেক্সট টোকেন কখনোই DB-তে থাকে না, কেবল SHA-256 হ্যাশ স
    expires_at TIMESTAMPTZ NOT NULL,
    consumed_at TIMESTAMPTZ NULL, -- একবার ব্যবহার হলে টাইমস্ট্যাম্প বসবে
    created_at TIMESTAMPTZ NOT NULL DEFAULT CURRENT_TIMESTAMP
);

CREATE INDEX idx_tokens_lookup ON email_verification_tokens(token_hash, consumed_at, expires_at)

-- ৫. অডিট লগ টেবিল (Security & Compliance)
CREATE TABLE user_registration_logs (
    id BIGSERIAL PRIMARY KEY,
    user_id UUID REFERENCES users(id) ON DELETE SET NULL,
    ip_address INET NOT NULL,
    user_agent TEXT NOT NULL,
    registration_source VARCHAR(50) DEFAULT 'WEB_DIRECT',
    created_at TIMESTAMPTZ NOT NULL DEFAULT CURRENT_TIMESTAMP
);
```

৩. স্টেট মেশিন ও লাইফসাইকেল (State Machine Model)

ইউজারের একাউন্ট বিভিন্ন দশার মধ্য দিয়ে যায়। এটিকে স্টেট মেশিন হিসেবে হ্যান্ডেল করলে অনাকাঙ্ক্ষিত বাগ ও সিকিউরিটি হোল প্রতিরোধ করা যায়:

```
stateDiagram-v2
    [*] ⟶ PENDING_VERIFICATION : রেজিস্ট্রেশন সম্পন্ন (Day 0)
    PENDING_VERIFICATION ⟶ ACTIVE : ইমেইল ভেরিফিকেশন লিংক ক্লিক
    PENDING_VERIFICATION ⟶ DELETED : ২৪ ঘণ্টার মধ্যে ভেরিফাই না করলে পার্জ (Purge)
    ACTIVE ⟶ SUSPENDED : পলিসি ভায়োলেশন / সিকিউরিটি লক
    SUSPENDED ⟶ ACTIVE : সাপোর্ট বা আপিল রিভিউ সফল
    ACTIVE ⟶ DEACTIVATED : ইউজার নিজেই একাউন্ট ডিলিট অনুরোধ করলে
```

স্টেট ট্রানজিশন রুলস:

1. `PENDING_VERIFICATION` : ইউজার লগইন করতে পারবেন, কিন্তু স্পর্শকাতর কর্মকাণ্ড (পোস্ট করা, টাকা লেনদেন, নোটিফিকেশন সাবস্ক্রিপশন) ব্লক থাকবে।
2. `ACTIVE` : সম্পূর্ণ অনুমোদিত সাধারণ মেম্বার।
3. `SUSPENDED` : কোনো স্প্যাম আক্রমণ বা ব্রুট-ফোর্স প্রচেষ্টার কারণে স্বয়ংক্রিয়ভাবে লক করা হয়েছে।

⬅️ *পূর্ববর্তী মডিউল* | 📖 *মূল সূচিপত্র (README)* | *পরবর্তী মডিউল* ➡️

মডিউল ০৩: সিকিউরিটি, ক্রিপ্টোগ্রাফি ও হার্ডেনিং (Security & Hardening)

⬅️ *পূর্ববর্তী মডিউল* | 📖 *মূল সূচিপত্র (README)* | *পরবর্তী মডিউল* ➡️

> ***লক্ষ্য:*** *পাসওয়ার্ড হ্যাশিংয়ের আধুনিক স্ট্যান্ডার্ড (Argon2id বনাম Bcrypt), টাইমিং অ্যাটাক প্রিভেনশন, সল্ট ও সিক্রেট পেপার এবং রেট লিমিটিং মেকানিজম বাস্তবায়ন করা।*

---

১. পাসওয়ার্ড হ্যাশিং অ্যালগরিদম: কেন Argon2id সেরা?

অনেক ডেভেলপার এখনও MD5, SHA-1 বা সাধারণ SHA-256 দিয়ে পাসওয়ার্ড হ্যাশ করার মতো মারাত্মক ভুল করেন। SHA ফ্যামিলি ডিজাইন করা হয়েছিল ডেটা ইন্টিগ্রিটি ও অতি দ্রুত হ্যাশ গণনার জন্য। আধুনিক GPU প্রতি সেকেন্ডে **কোটি কোটি SHA-256 হ্যাশ** ব্রুট-ফোর্স করতে পারে!

অ্যালগরিদম	মেমোরি হার্ডনেস	GPU/ASIC রেজিস্ট্যান্স	সাইড-চ্যানেল সুরক্ষা	প্রডাকশন সুপারিশ
MD5 / SHA-256	একদম নেই	০% (অত্যন্ত দুর্বল)	নেই	❌ সম্পূর্ণ নিষিদ্ধ
Bcrypt	খুব কম (৪ KB)	মাঝারি (CPU-bound)	ভালো	⚠️ গ্রহণযোগ্য (তবে ৭২ বাইট লিমিট আছে)
Argon2id	সর্বোচ্চ (কনফিগারযোগ্য)	**সর্বোচ্চ (Memory-hard)**	**সর্বোচ্চ (Hybrid resistance)**	✅ **স্বর্ণমান (Gold Standard)**

Argon2id প্যারামিটার কনফিগারেশন (OWASP সুপারিশকৃত):

- **Memory Cost (m):** `65536` KiB (64 MB)
- **Time Cost (t):** `3` Iterations
- **Parallelism (p):** `4` Threads

```
// Node.js এ argon2id হ্যাশিংয়ের প্রডাকশন এক্সাম্পল
import argon2 from 'argon2';

export async function hashPassword(plainPassword) {
    return await argon2.hash(plainPassword, {
        type: argon2.argon2id,
        memoryCost: 65536, // 64 MB
        timeCost: 3,
        parallelism: 4
    });
}

export async function verifyPassword(hash, plainPassword) {
    return await argon2.verify(hash, plainPassword);
}
```

---

২. ক্রিপ্টোগ্রাফিক সল্ট (Salt) এবং সার্ভার পেপার (Pepper)

1. **সল্ট (Salt):** প্রতিটি ইউজারের জন্য স্বয়ংক্রিয়ভাবে তৈরি হওয়া অনন্য র‍্যান্ডম বাইটস (Argon2id ও Bcrypt স্বয়ংক্রিয়ভাবে হ্যাশের ভেতর সল্ট সংরক্ষণ করে)। এটি রেইনবো টেবিল অ্যাটাক সম্পূর্ণ ধ্বংস করে।
2. **সার্ভার পেপার (Pepper):** একটি গোপন মাস্টার কি (Secret Key) যা ডেটাবেজে **কখনোই থাকে না**। এটি অ্যাপ্লিকেশনের এনভায়রনমেন্ট ভেরিয়েবলে (`APP_PEPPER`) অথবা ক্লাউড HSM/KMS-এ সংরক্ষিত থাকে।
    - পাসওয়ার্ড হ্যাশ করার আগে পেপার দিয়ে HMAC তৈরি করা হয়: $$ ext{Pre-Hash} = ext{HMAC-SHA256}(ext{Password}, ext{APP_PEPPER})$$
    - এরপর প্রি-হ্যাশকে Argon2id দিয়ে হ্যাশ করা হয়।
    - **ফলাফল:** হ্যাকার যদি ডেটাবেজ ডাম্প সম্পূর্ণ চুরিও করে নেয়, সার্ভার পেপার ছাড়া একটি পাসওয়ার্ডও ক্র্যাক করতে পারবে না।

---

৩. টাইমিং অ্যাটাক প্রিভেনশন (Timing-Safe Comparison)

সাধারণ স্ট্রিং কম্প্যারিজন (`string1 === string2`) প্রথম অমিল পাওয়ার সাথে সাথেই এক্সিকিউশন বন্ধ করে দেয়। হ্যাকাররা মাইক্রোসেকেন্ড নিখুঁতভাবে পরিমাপ করে টোকেন অনুমান করতে পারে (Side-Channel Timing Attack)।

ক্রিপ্টোগ্রাফিক টোকেন ও সিগনেচার মেলানোর সময় সর্বদা **কনস্ট্যান্ট-টাইম কম্প্যারিজন** ব্যবহার করতে হবে:

```
import crypto from 'crypto';

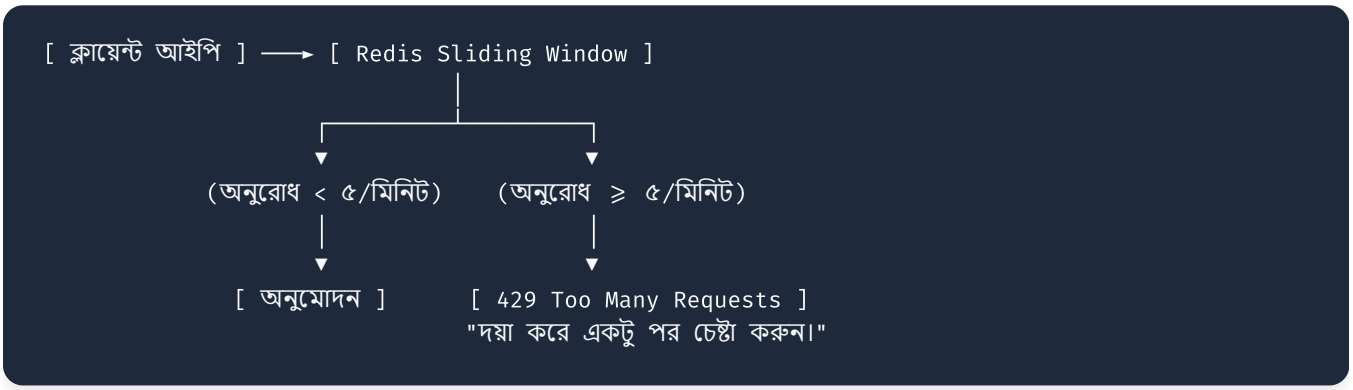
// ক্ষতিকর পদ্ধতি:
// if (userProvidedToken === dbToken) { ... } // ❌ টাইমিং অ্যাটাকে আক্রান্ত হতে পারে!

// নিরাপদ পদ্ধতি:
const isMatch = crypto.timingSafeEqual(
    Buffer.from(userProvidedHash, 'hex'),
    Buffer.from(storedHash, 'hex')
); // ✅ সর্বদা সমান সময় নেয়
```

---

৪. ব্রুট-ফোর্স ও রেজিস্ট্রেশন ফ্লাড প্রতিরোধে রেট লিমিটিং (Rate Limiting)

বোট ও হ্যাকাররা বটনেট দিয়ে হাজার হাজার ভুয়া একাউন্ট তৈরি করে সিস্টেম ক্র্যাশ করাতে পারে। Redis-ভিত্তিক **Sliding Window Counter** অ্যালগরিদম দিয়ে আমরা রেজিস্ট্রেশন সুরক্ষিত রাখব:



- **আইপি ভিত্তিক লিমিট:** প্রতি আইপি থেকে সর্বোচ্চ ৫টি রেজিস্ট্রেশন প্রতি ১০ মিনিটে।
- **সাবনেট লিমিট:** একই `/24` IPv4 রেঞ্জ থেকে সর্বোচ্চ ২০টি রিকোয়েস্ট প্রতি ঘণ্টায়।

> ⬅️ *পূর্ববর্তী মডিউল* | 📖 *মূল সূচিপত্র (README)* | *পরবর্তী মডিউল* ➡️

মডিউল ০৪: ভেরিফিকেশন মেকানিজম – ইমেইল বনাম ফোন (Email vs Phone OTP)

⬅️ *পূর্ববর্তী মডিউল* | 📖 *মূল সূচিপত্র (README)* | *পরবর্তী মডিউল* ➡️

> ***লক্ষ্য:*** *আর্থিক ও প্রযুক্তিগত দৃষ্টিকোণ থেকে ইমেইল ভেরিফিকেশন বনাম SMS OTP-এর গভীর বিশ্লেষণ এবং নিরাপদ টোকেন লাইফসাইকেল নিশ্চিত করা।*

---

১. আর্থিক ও অপারেশনাল সমীকরণ: কেন আমরা শুরুতে SMS বাদ দিচ্ছি?

একটি স্টার্টআপ বা ডেভেলপার প্রজেক্টের শুরুতে ফোন নাম্বার বাধ্যতামূলক করা সবচেয়ে বড় ভুলগুলোর একটি। আসুন বাস্তব আর্থিক চিত্রটি বিশ্লেষণ করি:

১.১ বাংলাদেশে SMS গেটওয়ের বাস্তব খরচ (SMS Cost Math)

বাংলাদেশে অনুমোদিত টেলকো অ্যাগ্রিগেটরদের (যেমন: SSL Wireless, Infobip, BulkSMSBD, Grameenphone/Robi Business) বর্তমান খরচ:

- **প্রতি SMS রেট:** ৳০.৩০ থেকে ৳০.৫৫ (ভ্যাট ও ট্যাক্সসহ)।
- **মাসিক ন্যূনতম পোর্টাল বা সার্ভিস ফি:** ৳২,০০০ থেকে ৳৫,০০০।
- **টেলকো মাস্কিং (Sender ID Approval):** BTRC ক্লিয়ারেন্স ও কর্পোরেট ডকুমেন্টে ১-২ মাস সময় এবং সেটআপ ফি।

```
রেজিস্ট্রেশন সংখ্যা        মাসিক SMS খরচ (গড়ে ৳০.৪৫/OTP)
──────────────────────      ──────────────────────────────
১,০০০ ইউজার                ৳৪৫০
১০,০০০ ইউজার               ৳৪,৫০০
৫০,০০০ ইউজার               ৳২২,৫০০
১,০০,০০০ ইউজার             ৳৪৫,০০০ - ৳৫৫,০০০ প্রতি মাসে!
```

> ⚠️ **WARNING (সতর্কবার্তা)**
> বোট বা ক্ষতিকর আক্রমণকারীরা যদি আপনার রেজিস্ট্রেশন ফর্মে "SMS Bombing" স্ক্রিপ্ট চালায়, তবে আপনার ক্রেডিট কার্ড বা অ্যাকাউন্টের কয়েক লাখ টাকা মাত্র কয়েক ঘণ্টায় নিঃশেষ হয়ে যেতে পারে!

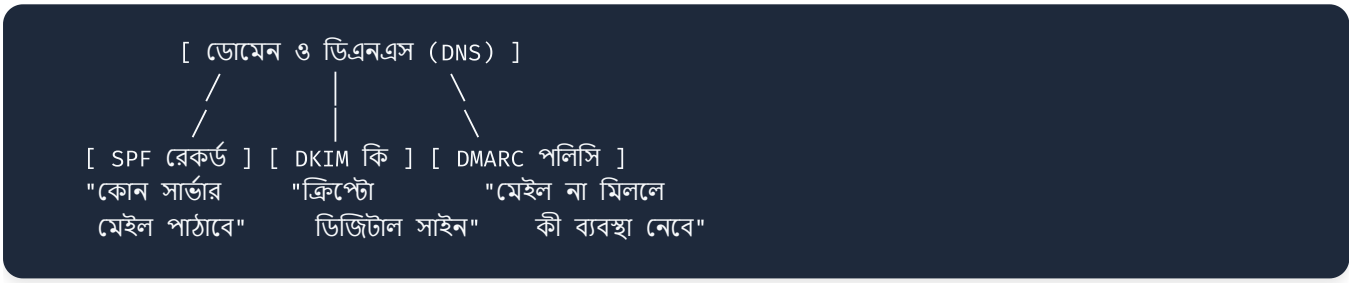
১.২ ইমেইল ভেরিফিকেশনের জিরো-কস্ট সুবিধা

- **সেলফ-হোস্টেড SMTP (Postfix / Haraka / Docker Mailserver):** খরচ **৳০ (সম্পূর্ণ ফ্রি)**।
- **ক্লাউড ট্রানজ্যাকশনাল মেইল সার্ভিস (AWS SES / Resend / Brevo / Mailgun):**
  - AWS SES: প্রতি ১,০০০ ইমেইলে মাত্র $০.১০ (১০ সেন্ট ~ ৳১২)।
  - Resend / Brevo: প্রতি মাসে ৩,০০০ থেকে ৯,০০০ ফ্রি ট্রানজ্যাকশনাল ইমেইল দেয়।
- **উপসংহার:** ডে ০-তে ইমেইল ভেরিফিকেশন বেছে নেওয়ার ফলে প্রাথমিক অপারেশনাল খরচ শূন্যের কোঠায় নেমে আসে, প্ল্যাটফর্ম সুরক্ষিত থাকে এবং কোনো আর্থিক রক্তক্ষরণ ছাড়াই লক্ষাধিক ইউজার অনবোর্ড করা সম্ভব হয়।

---

২. ইমেইল ডেলিভারেবিলিটি ট্রায়াঙ্গেল (SPF, DKIM, DMARC)

ইমেইল যাতে ইউজারের স্প্যাম ফোল্ডারে না গিয়ে সরাসরি **ইনবক্সে** পৌঁছায়, সেজন্য তিনটি DNS রেকর্ড সঠিকভাবে কনফিগার করতে হয়:



1. **SPF (Sender Policy Framework):** ডিএনএস টিএক্সটি (TXT) রেকর্ড যা বিশ্বকে জানায় কোন কোন আইপি এড্রেস আপনার ডোমেইন থেকে বৈধভাবে মেইল পাঠাতে পারে।
   উদাহরণ: `v=spf1 ip4:185.120.45.10 include:_spf.zengfy.com ~all`
2. **DKIM (DomainKeys Identified Mail):** প্রতিটি পাঠানো ইমেইলের হেডারে প্রাইভেট কি দিয়ে ক্রিপ্টোগ্রাফিক সাইন থাকে। প্রাপক মেইল সার্ভার পাবলিক কি দিয়ে যাচাই করে যে পথিমধ্যে মেইলটি কেউ বদল করেছে কি না।
3. **DMARC (Domain-based Message Authentication, Reporting, and Conformance):** রিসিভারকে নির্দেশ দেয় যদি SPF বা DKIM ফেইল করে, তবে মেইলটি রিজেক্ট বা কোয়ারেন্টাইন করতে হবে।

---

৩. স্টেটলেস ক্রিপ্টোগ্রাফিক ভেরিফিকেশন লিংক আর্কিটেকচার

ইউজারকে পাঠানো ভেরিফিকেশন লিংকের আর্কিটেকচার অত্যন্ত গুরুত্বপূর্ণ:

```
https://zengfy.top/verify-email?id=UUID&expires=1792323553&signature=HMAC_SHA256
```

কেন প্লেইনটেক্সট টোকেন ডেটাবেজে রাখা নিষেধ?

অনেক ডেভেলপার ডেটাবেজে `verify_token` কলামে প্লেইনটেক্সট স্ট্রিং রেখে দেন। কোনো দুর্বলতার কারণে ডেটাবেজ লিক হলে আক্রমণকারী সকলের টোকেন পেয়ে ভুয়া ভেরিফিকেশন করে ফেলতে পারে।

সঠিক ডিজাইন:

1. ইউজারের জন্য একটি ক্রিপ্টোগ্রাফিক হাই-এনট্রপি র‍্যান্ডম টোকেন তৈরি করুন:

```
const rawToken = crypto.randomBytes(32).toString('hex'); // ৬৪ অক্ষরের নিরাপদ টোকেন
```

2. ডেটাবেজে শুধুমাত্র এর **SHA-256 হ্যাশ** সংরক্ষণ করুন:

```
const tokenHash = crypto.createHash('sha256').update(rawToken).digest('hex');
// DB-তে tokenHash সেভ হবে, rawToken কখনোই DB-তে যাবে না!
```

3. ইউজারকে লিংকে `rawToken` পাঠানো হবে। যখন ইউজার ক্লিক করবেন, ব্যাকএন্ড আবার সেটিকে SHA-256 হ্যাশ করে ডেটাবেজের সাথে মিলাবে।
4. টোকেনের মেয়াদ থাকবে সর্বোচ্চ **২৪ ঘণ্টা** এবং একবার ভেরিফাই হলে টোকেনটি সাথে সাথে এক্সপায়ার (`consumed_at = NOW()`) হয়ে যাবে।

⬅️ *পূর্ববর্তী মডিউল* | 📖 *মূল সূচিপত্র (README)* | *পরবর্তী মডিউল* ➡️

মডিউল ০৫: রিয়েল-ওয়ার্ল্ড এজ কেস ও প্রডাকশন বাগস (Edge Cases & Production Bugs)

⬅️ *পূর্ববর্তী মডিউল* | 📖 *মূল সূচিপত্র (README)* | *পরবর্তী মডিউল* ➡️

> ***লক্ষ্য:*** *প্রডাকশনে লাখ লাখ ইউজার রেজিস্ট্রেশন করার সময় যেসব মারাত্মক বাগ ও ট্র্যাপের সম্মুখীন হতে হয়, সেগুলোর কার্যকর ইঞ্জিনিয়ারিং সমাধান জানা।*

---

১. কনকারেন্সি এবং রেস কন্ডিশন (Double Submit Race Condition)

ধরা যাক, একজন ইউজার রেজিস্ট্রেশন ফর্মে সাবমিট বাটনে দ্রুত দুইবার ক্লিক করলেন (Double-click)।

```
থ্রেড ১ ──► [ ইমেইল কি আছে? ] ──► (না, নেই) ──► [ ইউজার ইনসার্ট ] ──► (সফল)
                                ▲
থ্রেড ২ ────────────────────────┘ একই মিলি-সেকেন্ডে চেক করে দেখল "নেই"! ──► [ ডুপ্লিকেট ইনসার্ট! ]
```

সমাধান:

1. **ফ্রন্টএন্ড স্তরে:** বাটনে ক্লিক করার সাথে সাথে বাটন `disabled` করা এবং লোডিং স্পিনার দেখানো।
2. **ডেটাবেজ স্তরে:** `email` কলামের উপর বাধ্যতামূলক `UNIQUE CONSTRAINT` থাকা আবশ্যক।
3. **সফটওয়্যার কোড স্তরে:** ডেটাবেজের ইউনিক কি ভায়োলেশন এরর কোড (`23505` PostgreSQL-এ) ক্যাচ করে সুন্দরভাবে ক্লায়েন্টকে জানানো: "এই ইমেইলটি দিয়ে ইতোমধ্যে অ্যাকাউন্ট তৈরি করা হয়েছে।"

---

২. ইউনিকোড ও আন্তর্জাতিকীকরণ সমস্যা (Unicode Homograph Attacks)

আক্রমণকারীরা একই দেখতে কিন্তু ভিন্ন ক্যারেক্টার দিয়ে ক্ষতিকর ডোমেইন বা ইমেইল স্পুফ করতে পারে।

- ল্যাটিন ইংরেজি অক্ষর `a` (ASCII `0×61`)
- সিরিলিক রাশিয়ান অক্ষর `а` (Unicode `U+0430`)

উভয় অক্ষর দেখতে হুবহু একই রকম (`a`), কিন্তু কম্পিউটারের জন্য দুটি সম্পূর্ণ ভিন্ন ক্যারেক্টার!

সমাধান:

ইমেইল এবং নাম ডেটাবেজে সেভ করার পূর্বে অবশ্যই **Unicode NFKC Normalization** চালাতে হবে:

```
const cleanEmail = rawEmail.trim().toLowerCase().normalize('NFKC');
```

---

৩. প্লাস-ট্যাগিং ও ডট-ইনসেনসিটিভিটি (Plus-Addressing & Dot-Tricks)

গুগল জিমেইল অ্যাকাউন্টে একটি বিশেষ সুবিধা আছে:

- `sazzad.zengfy@gmail.com`
- `s.a.z.z.a.d.zengfy@gmail.com`
- `sazzad.zengfy+freecredit@gmail.com`

গুগলের কাছে এই তিনটি ঠিকানাই এক। কিন্তু সাধারণ ব্যাকএন্ড এদের তিনটি ভিন্ন ইমেইল হিসেবে গণ্য করে। এর ফলে অসাধু ইউজাররা ট্রায়াল ক্রেডিট বা ফ্রি কুপন অপব্যবহার করার জন্য শত শত একাউন্ট খুলে ফেলে।

সমাধান:

রেজিস্ট্রেশনের সময় একটি ক্যানোনিক্যাল ইমেইল (Canonical Email) প্রসেসিং ফাংশন ব্যবহার করা:

```
export function canonicalizeEmail(email) {
    let [localPart, domain] = email.trim().toLowerCase().split('@');
    if (domain === 'gmail.com' || domain === 'googlemail.com') {
        localPart = localPart.replace(/\./g, ''); // সব ডট মুছে ফেলা
        localPart = localPart.split('+')[0];      // প্লাস এবং তার পরের সব অক্ষর বাদ দেওয়া
    }
    return `${localPart}@${domain}`;
}
```

---

৪. ডিসপোজেবল টেম্পোরারি ইমেইল প্রতিরোধ (Disposable Email Filtering)

অনেকে `10minutemail`, `temp-mail.org`, `guerrillamail` এর মতো ক্ষতিকর অস্থায়ী মেইল সার্ভিস দিয়ে ভুয়া একাউন্ট খুলে স্প্যামিং করে।

সমাধান:

রেজিস্ট্রেশন ভ্যালিডেশনের সময় একটি ওপেন-সোর্স ডিসপোজেবল ডোমেইন ব্ল্যাকলিস্ট দিয়ে চেক করতে হবে:

```
const DISPOSABLE_DOMAINS = new Set([
    'tempmail.com', '10minutemail.com', 'mailinator.com',
    'guerrillamail.com', 'sharklasers.com', 'yopmail.com'
]);

export function isDisposableEmail(domain) {
    return DISPOSABLE_DOMAINS.has(domain.toLowerCase());
}
```

⬅️ *পূর্ববর্তী মডিউল* | 📖 *মূল সূচিপত্র (README)* | *পরবর্তী মডিউল* ➡️

মডিউল 06 / ১১

মডিউল ০৬: ইন্ডাস্ট্রি কেস স্টাডি ও বাস্তব বিশ্লেষণ (Industry Case Studies)

> ⬅️ *পূর্ববর্তী মডিউল* | 📖 *মূল সূচিপত্র (README)* | *পরবর্তী মডিউল* ➡️

> ***লক্ষ্য:*** *গুগল, গিটহাব, স্ট্রাইপ এবং দেশীয় বিকাশ/নগদের সফল সিস্টেমগুলোর বাস্তব বিশ্লেষণ থেকে শিক্ষা নেওয়া।*

---

১. কেস স্টাডি ০১: গুগল অ্যাকাউন্টস (Google Glif Flow)

গুগলের লগইন ও সাইন-আপ ইউআরএলটি লক্ষ্য করুন:

```
https://accounts.google.com/v3/signin/identifier?flowName=GlifWebSignIn&flowEntry=ServiceLogin&d
```

পেছনের ইঞ্জিনিয়ারিং কৌশল:

1. **Identifier-First Architecture:** গুগল কখনো একই স্ক্রিনে ইমেইল ও পাসওয়ার্ড একসাথে চায় না। প্রথমে চায় আইডেন্টিফায়ার (ইমেইল বা ফোন)। এর সুবিধা:
   - আইডেন্টিফায়ার টাইপ করার পর ব্যাকএন্ড সিদ্ধান্ত নেয় ইউজারকে পাসওয়ার্ড চাওয়া হবে, নাকি সিকিউরিটি কি (FIDO2 WebAuthn), নাকি পাসকি (Passkey), নাকি SSO রিডাইরেক্ট দেওয়া হবে।
2. **Glif Web Sign-In:** এটি গুগলের আধুনিক একক অথেনটিকেশন ফ্রেমওয়ার্ক। এটি ব্যাকএন্ড স্টেট মেশিনের সাথে কানেক্টেড।
3. `dsh` **ও সেশন সিগনেচার:** কোনো ইউজার একই পেজে ১০ মিনিট বসে থাকলে বা রিফ্রেশ করলে সেশনের ডেটা যাতে টেম্পার না হয়, সেজন্য স্টেট প্যারামিটার এনকোড করা থাকে।

---

২. কেস স্টাডি ০২: গিটহাব (GitHub)

গিটহাবের নতুন রেজিস্ট্রেশন ফ্লোটি সম্পূর্ণ মিনিমালিস্ট এবং টার্মিনাল স্টাইল:

1. প্রথমে শুধুমাত্র **ইমেইল** চাওয়া হয়।
2. এরপর ডায়নামিক মসৃণ অ্যানিমেশনে **পাসওয়ার্ড** ফিল্ড নেমে আসে।

3. এরপর **ইউজারনেম** চাওয়া হয়।
4. এরপর সাথে সাথে একাউন্ট তৈরি হয়ে যায় এবং ইনবক্সে ৬-সংখ্যার কোড পাঠিয়ে ভেরিফাই করতে বলা হয়।
5. **ফলাফল:** গিটহাবে একজন ডেভেলপার মাত্র ৩০ সেকেন্ডের মধ্যে একাউন্ট তৈরি করে প্রথম রিপোজিটরি ক্লোন করতে পারেন।

---

৩. কেস স্টাডি ০৩: স্ট্রাইপ (Stripe - Global FinTech Standard)

বিশ্ববিখ্যাত পেমেন্ট গেটওয়ে স্ট্রাইপ বিলিয়ন ডলার লেনদেন পরিচালনা করে। কিন্তু তাদের রেজিস্ট্রেশন ফর্মটি দেখুন:

- শুধুমাত্র **Email, Full Name, Password এবং Country**।
- কোনো ট্রেড লাইসেন্স বা ব্যাংক স্টেটমেন্ট রেজিস্ট্রেশন ফর্মে চাওয়া হয় না!
- ইউজার সাইন-আপ করে ড্যাশবোর্ডে ঢোকেন, টেস্ট মোডে API কোড চালান। যখন তিনি আসল টাকা তুলতে যাবেন, তখন তিনি **প্রগ্রেসিভ KYC** পূরণ করেন।

---

৪. কেস স্টাডি ০৪: বাংলাদেশের প্রযুক্তি ক্ষেত্র (বিকাশ, নগদ, পাঠাও)

অনেকে প্রশ্ন করেন: "বিকাশ বা নগদ তো প্রথমেই ফোন নাম্বার ও এনআইডি চায়, আমরা কেন করব না?"

বাস্তব কারণ ও পার্থক্য:

1. **MFS রেগুলেশন:** বিকাশ ও নগদ বাংলাদেশ ব্যাংকের কড়া রেগুলেশনের (AML/CFT আইন) আওতায় মোবাইল ফাইন্যান্সিয়াল সার্ভিস পরিচালনা করে। আইন অনুযায়ী তাদের প্রথম কদমেই জাতীয় পরিচয়পত্র (NID) ভেরিফিকেশন বাধ্যতামূলক।
2. **ভিন্ন বিজনেস মডেল:** Zengfy কোনো ব্যাংক নয়। Zengfy হলো সিভিক ও সোশ্যাল টেকনোলজি প্ল্যাটফর্ম।
3. সামাজিক ও প্রযুক্তিগত প্ল্যাটফর্মে যদি প্রথমেই এনআইডি বা ফোন চাওয়া হয়, তবে ৯৫% ইউজার অনবোর্ড হওয়ার আগেই প্ল্যাটফর্ম ছেড়ে চলে যাবে। তাই আমাদের মডেল হবে: **Day 0 তে ৩টি ফিল্ড এবং পরবর্তীতে স্বপ্রণোদিত সিভিক ব্যাজ প্রদান**।

⬅️ *পূর্ববর্তী মডিউল* | 📖 *মূল সূচিপত্র (README)* | *পরবর্তী মডিউল* ➡️

মডিউল ০৭: Zengfy ইমপ্লিমেন্টেশন ব্লুপ্রিন্ট (Zengfy Sovereign Blueprint)

⬅️ *পূর্ববর্তী মডিউল* | 📖 *মূল সূচিপত্র (README)* | *পরবর্তী মডিউল* ➡️

> ***লক্ষ্য:*** *Zengfy ইকোসিস্টেমের স্পেসিফিক রিকোয়ারমেন্ট অনুযায়ী রেজিস্ট্রেশন সিস্টেমের বাস্তবায়ন রোডম্যাপ এবং সার্বভৌম ট্রাস্ট ব্যাজ আর্কিটেকচার চূড়ান্ত করা।*

---

১. সার্বভৌম ট্রাস্ট টিয়ার মডেল (Zengfy Trust Tiers)

Zengfy ইকোসিস্টেমে সিটিজেনদের আস্থা ও অধিকারের ৪টি স্তর থাকবে:

```
[ Tier 3: Sovereign Civic Citizen ] ◄── এনআইডি বা লোকাল ওয়ার্ড ভেরিফিকেশন (ভোট ও আইন)
                ▲
[ Tier 2: Community Verified ]      ◄── ফোন নাম্বার যুক্ত ও ২ জন ভেরিফাইড নাগরিকের অনুমোদন
                ▲
[ Tier 1: Email Verified Member ]   ◄── ইমেইল ভেরিফিকেশন লিংক সম্পন্ন (পোস্ট, কমেন্ট)
                ▲
[ Tier 0: Guest / Unverified ]       ◄── ৩-ফিল্ড সাইন-আপ সম্পন্ন (শুধু রিড এক্সেস)
```

টিয়ার	নাম	শর্ত	অর্জিত অধিকার ও সুযোগ-সুবিধা
Tier 0	Guest Citizen	নাম, ইমেইল, পাসওয়ার্ড দিয়ে সাইন-আপ	প্ল্যাটফর্ম ভিজিট করা, কনটেন্ট পড়া, ডিরেক্টরি সার্চ করা।
Tier 1	Verified Citizen	ইমেইল ভেরিফিকেশন সম্পন্ন	সোশ্যাল ফিডে পোস্ট করা, কমেন্ট করা, কমিউনিটি নোটিফিকেশন পাওয়া।
Tier 2	Community Anchor	একাউন্ট সেন্টারে ফোন নাম্বার ভেরিফাই	লোকাল নেইবারহুড গ্রুপ তৈরি করা, সিভিক বিপদে অ্যালার্ট ইস্যু করা।
Tier 3	Sovereign Leader	এনআইডি / স্মার্ট কার্ড ডিজিটাল প্রুফ	অফিসিয়াল সিভিক ভোট প্রদান, নীতি নির্ধারণ ও ডিজিটাল পিটিশন তৈরি।

২. মডুলার আর্কিটেকচার রোডম্যাপ

রেজিস্ট্রেশন মডিউলটি Zengfy Core-এ একটি সম্পূর্ণ পৃথক এবং স্বাধীন মডিউল (`AuthModule`) হিসেবে পরিচালিত হবে:

```
P:\Zengfy-Ecosystem\core├── modules│   └── auth│       ├── controllers\    # RegistrationControl
│       ├── services\       # RegistrationService, TokenService, PasswordHasher
│       ├── validators\     # RegisterRequestValidator
│       ├── models\         # User, EmailVerificationToken
│       └── views\          # responsive-registration.blade.php / react-form.tsx
```

এই ডিকাপল্ড আর্কিটেকচারের ফলে ভবিষ্যতে যখন Zengfy Social, Zengfy Law বা অন্যান্য সিভিক মডিউল যুক্ত হবে, তারা সেন্ট্রাল অথেনটিকেশন সার্ভিসের সাথে একক সাইন-অন (Single Sign-On - SSO) প্রটোকলে যুক্ত হতে পারবে।

⬅️ পূর্ববর্তী মডিউল | 📖 মূল সূচিপত্র (README) | পরবর্তী মডিউল ➡️

মডিউল ০৮: আইটি কোম্পানির ভূমিকা ও টিম ওয়ার্কফ্লো (IT Company Roles & Legal Workflow)

⬅️ *পূর্ববর্তী মডিউল* | 📖 *মূল সূচিপত্র (README)* | *পরবর্তী মডিউল* ➡️

> ***লক্ষ্য:*** *একটি পূর্ণাঙ্গ প্রফেশনাল সফটওয়্যার কোম্পানিতে রেজিস্ট্রেশন ফিচার বাস্তবায়নের সময় কার কী দায়িত্ব এবং বাংলাদেশের সাইবার নিরাপত্তা ও ডেটা সুরক্ষা আইনের কমপ্লায়েন্স নিশ্চিত করা।*

---

১. আইটি কোম্পানির বিভিন্ন রোল ও দায়িত্ব (RACI Matrix)

একটি পরিপক্ক ইঞ্জিনিয়ারিং সংস্থায় রেজিস্ট্রেশন কেবল ডেভেলপারের কাজ নয়। প্রতিটি রোলের নির্দিষ্ট সীমানা থাকে:

পদবী (Role)	মূল দায়িত্ব ও কাজের পরিধি	সফলতার মানদণ্ড (KPI)
১. প্রোডাক্ট ম্যানেজার (PM)	বিজনেসের চাহিদা বোঝা, ৩-ফিল্ড সাইন-আপের পিআরডি (PRD) ও ইউজার স্টোরি লেখা।	কনভার্সন রেট ৯০%+ রাখা এবং সাইন-আপ ড্রপ-অফ হ্রাস।
২. ইউআই/ইউএক্স ডিজাইনার	ফর্ম ডিজাইন, মোবাইল স্ক্রিনে স্পর্শের সুবিধা (Touch Target 48px), পাসওয়ার্ড স্ট্রেন্থ মিটার ও ত্রুটির মেসেজ।	SUS স্কোর ৮৫+, ফর্ম পূরণের গড় সময় < ৪৫ সেকেন্ড।
৩. লিগ্যাল ও সিআইএসও (Legal & CISO)	বাংলাদেশের সাইবার নিরাপত্তা আইন ২০২৩, প্রাইভেসি পলিসি ও ডেটা কনসেন্ট অডিট।	জিরো লিগ্যাল লায়াবিলিটি, GDPR ও দেশীয় আইনের ১০০% কমপ্লায়েন্স।
৪. ব্যাকএন্ড আর্কিটেক্ট	ডেটাবেজ স্কিমা, ট্রানজ্যাকশন, Argon2id হ্যাশিং, রেট লিমিটিং ও কিউ আর্কিটেকচার।	API লেটেন্সি < ৫০ms, জিরো রেস কন্ডিশন।
৫. ফ্রন্টএন্ড ইঞ্জিনিয়ার	রেসপনসিভ টেলউইন্ড ফর্ম, ক্লায়েন্ট-সাইড ভ্যালিডেশন, অ্যাক্সেসিবিলিটি (ARIA)।	লাইটহাউস স্কোর ৯৫+, মোবাইল লোড টাইম < ১ সেকেন্ড।
৬. কিউএ ইঞ্জিনিয়ার (QA)	পজিটিভ/নেগেটিভ টেস্ট কেস, বাউন্ডারি টেস্ট, সিকিউরিটি পেন-টেস্টিং।	জিরো ক্রিটিক্যাল বাগ প্রডাকশন রিলিজের পূর্বে।
৭. ডেভঅপস ও এসআরই (DevOps)	ডকারাইজেশন, সিআই/সিডি পাইপলাইন, ক্লাউডফ্লেয়ার ডব্লিউএএফ রুলস ও মনিটরিং।	৯৯.৯৯% আপটাইম, স্বয়ংক্রিয় ডেব ফ্ল্যাগ অ্যালার্ট।

---

২. আইনি ফ্রেমওয়ার্ক: বাংলাদেশের সাইবার নিরাপত্তা আইন ২০২৩ ও ডেটা সুরক্ষা

বাংলাদেশে যে কোনো অনলাইন প্ল্যাটফর্ম পরিচালনার ক্ষেত্রে স্থানীয় আইনের সুস্পষ্ট জ্ঞান থাকা আবশ্যক:

২.১ সাইবার নিরাপত্তা আইন ২০২৩ (Cyber Security Act 2023) এর গুরুত্বপূর্ণ ধারা

1. **নাগরিক তথ্যের গোপনীয়তা:** ইউজারের ব্যক্তিগত তথ্য (নাম, ইমেইল, পাসওয়ার্ড) যথাযথ নিরাপত্তা ব্যবস্থা ছাড়া সংরক্ষণ করা দণ্ডনীয় অপরাধ।
2. **অবৈধ অনুপ্রবেশ ও ব্রুট-ফোর্স প্রতিরোধ:** সিস্টেমে ব্রুট-ফোর্স বা হ্যাকিংয়ের কোনো চেষ্টা হলে তার আইপি ও অডিট লগ অন্তত **৬ মাস থেকে ১ বছর** সংরক্ষণ করে রাখা প্রয়োজন, যাতে আইনি তদন্তে সহায়তা দেওয়া যায়।
3. **অনুমতিহীন তথ্য আদান-প্রদান নিষিদ্ধ:** রেজিস্ট্রেশন ফর্মে স্পষ্টভাবে একটি চেকবক্স বা টার্মস অফ সার্ভিস ডিসক্লেমার থাকতে হবে:
   "রেজিস্ট্রেশন বাটনে ক্লিক করার মাধ্যমে আপনি আমাদের ব্যবহারের শর্তাবলী এবং গোপনীয়তা নীতিতে সম্মতি দিচ্ছেন।"

২.২ ডেটা মিনিমাইজেশন প্রিন্সিপাল (Data Minimization)

বিশ্বব্যাপী প্রাইভেসি আইনের মূল ভিত্তি হলো: **"আপনার প্রজেক্টের কাজের জন্য যতটুকু তথ্য না হলেই নয়, ঠিক ততটুকুই সংগ্রহ করুন।"**

- যেহেতু প্রথম দিন আমাদের ফোন নাম্বার বা এনআইডির কোনো প্রয়োজন নেই, তাই তা সংগ্রহ না করাই আইনি দিক থেকে সবচেয়ে নিরাপদ।
- যত কম সংবেদনশীল তথ্য সিস্টেমে থাকবে, ডেটা ব্রিচ বা ফাঁসের ঝুঁকি তত কম থাকবে।

⬅️ *পূর্ববর্তী মডিউল* | 📖 *মূল সূচিপত্র (README)* | *পরবর্তী মডিউল* ➡️

মডিউল ০৯: ডে ০ রুট সিনারিও ইমপ্লিমেন্টেশন গাইড (Step-by-Step Code Guide)

⬅️ *পূর্ববর্তী মডিউল* | 📖 *মূল সূচিপত্র (README)* | *পরবর্তী মডিউল* ➡️

> ***লক্ষ্য:*** *ডে ০-এর জন্য প্রয়োজনীয় ৩টি ফিল্ডের (Full Name, Email, Password) ব্যাকএন্ড আর্কিটেকচার, ক্রিপ্টোগ্রাফিক ভেরিফিকেশন সার্ভিস এবং প্রিমিয়াম মোবাইল-ফ্রেন্ডলি ফ্রন্টএন্ড কোড তৈরি করা।*

---

১. ব্যাকএন্ড সার্ভিস লেয়ার (Node.js / Express / TypeScript)

আমরা ক্লিন আর্কিটেকচার বজায় রেখে কোড লিখব।

১.১ ভ্যালিডেশন রুলস (`validators/register.validator.ts`)

```
import { z } from 'zod';

// ডিসপোজেবল ডোমেইন তালিকা
const BLACKLISTED_DOMAINS = ['10minutemail.com', 'tempmail.com', 'mailinator.com', 'yopmail.com'

export const RegisterSchema = z.object({
  fullName: z
    .string()
    .min(2, { message: 'নাম কমপক্ষে ২ অক্ষরের হতে হবে।' })
    .max(100, { message: 'নাম ১০০ অক্ষরের বেশি হতে পারবে না।' })
    .trim(),
  email: z
    .string()
    .email({ message: 'একটি বৈধ ইমেইল ঠিকানা প্রদান করুন।' })
    .trim()
    .toLowerCase()
    .refine((email) => {
      const domain = email.split('@')[1];
      return !BLACKLISTED_DOMAINS.includes(domain);
    }, { message: 'অস্থায়ী বা ডিসপোজেবল ইমেইল দিয়ে একাউন্ট খোলা যাবে না।' }),
  password: z
    .string()
    .min(8, { message: 'পাসওয়ার্ড কমপক্ষে ৮ অক্ষরের হতে হবে।' })
    .regex(/[A-Z]/, { message: 'পাসওয়ার্ডে অন্তত একটি বড় হাতের অক্ষর (A-Z) থাকতে হবে।' })
    .regex(/[0-9]/, { message: 'পাসওয়ার্ডে অন্তত একটি সংখ্যা (0-9) থাকতে হবে।' })
});
```

১.২ রেজিস্ট্রেশন সার্ভিস (`services/auth.service.ts`)

```
import argon2 from 'argon2';
import crypto from 'crypto';
import { db } from '../config/database';

export class AuthService {
  /**
   * ডে ০ মিনিমালিস্ট রেজিস্ট্রেশন প্রসেস
   */
  async registerUser(data: { fullName: string; email: string; password: string }) {
    return await db.transaction(async (trx) => {
      // ১. ইমেইল পূর্বে ব্যবহৃত হয়েছে কি না চেক
      const existing = await trx('users').where({ email: data.email }).first();
      if (existing) {
        throw new Error('এই ইমেইল দিয়ে ইতোমধ্যে একাউন্ট খোলা হয়েছে।');
      }

      // ২. আধুনিক Argon2id দিয়ে পাসওয়ার্ড হ্যাশ করা
      const passwordHash = await argon2.hash(data.password, {
        type: argon2.argon2id,
        memoryCost: 65536, // 64 MB
        timeCost: 3,
        parallelism: 4
      });

      // ৩. ইউজার টেবিলে ডাটা ইনসার্ট করা
      const [newUser] = await trx('users')
        .insert({
          full_name: data.fullName,
          email: data.email,
          password_hash: passwordHash,
          status: 'PENDING_VERIFICATION'
        })
        .returning(['id', 'full_name', 'email']);

      // ৪. ক্রিপ্টোগ্রাফিক হাই-এনট্রপি ভেরিফিকেশন টোকেন তৈরি
      const rawToken = crypto.randomBytes(32).toString('hex');
      const tokenHash = crypto.createHash('sha256').update(rawToken).digest('hex');

      // ২৪ ঘণ্টার মেয়াদ নির্ধারণ
      const expiresAt = new Date(Date.now() + 24 * 60 * 60 * 1000);

      // ৫. হ্যাশ সংরক্ষণ
      await trx('email_verification_tokens').insert({
        user_id: newUser.id,
        token_hash: tokenHash,
        expires_at: expiresAt
      });

      // ৬. সিকিউর ভেরিফিকেশন ইউআরএল তৈরি (ইউজারের জন্য)
      const verifyUrl = `${process.env.APP_URL}/api/v1/auth/verify-email?token=${rawToken}&userI

      // ৭. ব্যাকগ্রাউন্ড কিউ-তে ইমেইল পাঠানোর ইভেন্ট পুশ করা
      await emailQueue.add('SendVerificationEmail', {
```

```
        to: newUser.email,
        name: newUser.full_name,
        verifyUrl: verifyUrl
      });

      return {
        success: true,
        message: 'রেজিস্ট্রেশন সফল হয়েছে! দয়া করে আপনার ইমেইল চেক করে একাউন্ট ভেরিফাই করুন।',
        user: { id: newUser.id, email: newUser.email }
      };
    });
  }

  /**
   * ইমেইল ভেরিফিকেশন টোকেন হ্যান্ডলার
   */
  async verifyEmailToken(userId: string, rawToken: string) {
    const tokenHash = crypto.createHash('sha256').update(rawToken).digest('hex');

    const tokenRecord = await db('email_verification_tokens')
      .where({
        user_id: userId,
        token_hash: tokenHash,
        consumed_at: null
      })
      .andWhere('expires_at', '>', new Date())
      .first();

    if (!tokenRecord) {
      throw new Error('ভেরিফিকেশন লিংকটি ভুল অথবা এর মেয়াদ শেষ হয়ে গেছে।');
    }

    await db.transaction(async (trx) => {
      // টোকেন কনজিউম করা
      await trx('email_verification_tokens')
        .where({ id: tokenRecord.id })
        .update({ consumed_at: new Date() });

      // ইউজারকে সক্রিয় করা
      await trx('users')
        .where({ id: userId })
        .update({
          status: 'ACTIVE',
          email_verified_at: new Date()
        });
    });

    return { success: true, message: 'অভিনন্দন! আপনার ইমেইল সফলভাবে ভেরিফাইড হয়েছে।' };
  }
}
```

---

২. ফ্রন্টএন্ড ফর্ম ইমপ্লিমেন্টেশন (Tailwind CSS + Mobile Responsive)

মোবাইল স্ক্রিনে যাতে এক ফোঁটাও স্ক্রল না করে এক ক্লিকে সাইন-আপ করা যায়, তার জন্য একটি প্রিমিয়াম ফর্ম:

```
<div class="min-h-screen bg-slate-50 flex items-center justify-center p-4 sm:p-6 font-sans">
  <div class="w-full max-w-md bg-white rounded-3xl shadow-xl border border-slate-100 p-6 sm:p-8"

    <!— হেডার —>
    <div class="text-center mb-6">
      <div class="w-12 h-12 bg-blue-600 rounded-2xl flex items-center justify-center mx-auto mb-
        <span class="text-white text-2xl font-black">Z</span>
      </div>
      <h2 class="text-2xl font-black text-slate-900 tracking-tight">নতুন একাউন্ট খুলুন</h2>
      <p class="text-xs text-slate-500 mt-1 font-medium">Zengfy Sovereign প্ল্যাটফর্মে আপনাকে স্বাগতম<
    </div>

    <!— গুগল ওয়ান-ট্যাপ অলটারনেটিভ বাটন —>
    <button type="button" class="w-full flex items-center justify-center gap-2.5 py-2.5 px-4 rou
      <svg class="w-4 h-4" viewBox="0 0 24 24"><path fill="#4285F4" d="M22.56 12.25c0-.78-.07-1.
      Google দিয়ে সরাসরি শুরু করুন
    </button>

    <div class="relative flex items-center justify-center my-4">
      <div class="border-t border-slate-200 w-full"></div>
      <span class="bg-white px-3 text-[11px] font-bold text-slate-400 uppercase tracking-wider">
    </div>

    <!— ফর্ম (কেবল ৩টি ফিল্ড) —>
    <form id="regForm" class="space-y-4">
      <!— ফিল্ড ১: পুরো নাম —>
      <div>
        <label for="fullName" class="block text-xs font-bold text-slate-700 mb-1">পুরো নাম (Full
        <input type="text" id="fullName" name="fullName" required placeholder="মোঃ সাজ্জাদ হোসেন"
               class="w-full px-3.5 py-2.5 rounded-xl border border-slate-200 text-xs text-slate
      </div>

      <!— ফিল্ড ২: ইমেইল ঠিকানা —>
      <div>
        <label for="email" class="block text-xs font-bold text-slate-700 mb-1">ইমেইল ঠিকানা (Emai
        <input type="email" id="email" name="email" required placeholder="name@example.com"
               class="w-full px-3.5 py-2.5 rounded-xl border border-slate-200 text-xs text-slate
      </div>

      <!— ফিল্ড ৩: পাসওয়ার্ড —>
      <div>
        <label for="password" class="block text-xs font-bold text-slate-700 mb-1">পাসওয়ার্ড (Passw
        <input type="password" id="password" name="password" required placeholder="কমপক্ষে ৮ অক্ষ
               class="w-full px-3.5 py-2.5 rounded-xl border border-slate-200 text-xs text-slate
        <!— লাইভ পাসওয়ার্ড স্ট্রেন্থ বার —>
        <div class="w-full bg-slate-100 h-1.5 rounded-full mt-2 overflow-hidden">
          <div id="strengthBar" class="h-full w-1/4 bg-red-500 transition-all duration-300"></di
        </div>
      </div>

      <!— সাবমিট বাটন —>
      <button type="submit" id="submitBtn"
              class="w-full py-3 px-4 bg-blue-600 hover:bg-blue-700 active:scale-[0.98] text-whi
        <span>একাউন্ট তৈরি করুন</span>
      </button>
```

```
      <!-- টার্মস ও লিগ্যাল নোটিশ -->
      <p class="text-[10px] text-center text-slate-400 mt-4 leading-relaxed">
        রেজিস্ট্রেশন সম্পন্ন করার মাধ্যমে আপনি আমাদের <a href="#" class="text-blue-600 underline">ব্যবহা
      </p>
    </form>

  </div>
</div>
```

> ⬅️ *পূর্ববর্তী মডিউল* | 📖 *মূল সূচিপত্র (README)* | *পরবর্তী মডিউল* ➡️

মডিউল ১০: টেস্টিং, কিউএ ও ডেভঅপস রানবুক (Testing, QA & DevOps Runbook)

⬅️ *পূর্ববর্তী মডিউল* | 📖 *মূল সূচিপত্র (README)*

লক্ষ্য: *অটোমেটেড ইউনিট টেস্ট, সিকিউরিটি পেন-টেস্টিং, লোকাল ডকার এনভায়রনমেন্ট এবং প্রডাকশন ইনসিডেন্ট রেসপন্স প্রস্তুত করা।*

---

১. অটোমেটেড ইন্টিগ্রেশন টেস্ট কেসেস (Jest / Vitest)

```
import request from 'supertest';
import { app } from '../app';
import { db } from '../config/database';

describe('Auth Registration Integration Tests', () => {
  beforeEach(async () => {
    await db('email_verification_tokens').truncate();
    await db('users').del();
  });

  test('পজিটিভ টেস্ট: বৈধ ৩টি ফিল্ড দিলে সফলভাবে 201 Created হবে', async () => {
    const res = await request(app)
      .post('/api/v1/auth/register')
      .send({
        fullName: 'সাজ্জাদ হোসেন',
        email: 'sazzad@zengfy.com',
        password: 'Password123'
      });

    expect(res.status).toBe(201);
    expect(res.body.success).toBe(true);
    expect(res.body.user.email).toBe('sazzad@zengfy.com');
  });

  test('নেগেটিভ টেস্ট: পূর্বে ব্যবহৃত ইমেইল দিলে 400 এরর দেবে', async () => {
    await request(app).post('/api/v1/auth/register').send({
      fullName: 'ইউজার ১',
      email: 'duplicate@zengfy.com',
      password: 'Password123'
    });

    const res = await request(app).post('/api/v1/auth/register').send({
      fullName: 'ইউজার ২',
      email: 'duplicate@zengfy.com',
      password: 'Password123'
    });

    expect(res.status).toBe(400);
    expect(res.body.message).toContain('ইতোমধ্যে একাউন্ট খোলা হয়েছে');
  });

  test('সিকিউরিটি টেস্ট: ডিসপোজেবল ইমেইল স্বয়ংক্রিয়ভাবে ব্লক করবে', async () => {
    const res = await request(app).post('/api/v1/auth/register').send({
      fullName: 'বোট ইউজার',
      email: 'spammer@10minutemail.com',
      password: 'Password123'
    });

    expect(res.status).toBe(400);
    expect(res.body.message).toContain('ডিসপোজেবল ইমেইল');
```

```
  });
});
```

---

২. সম্পূর্ণ লোকাল ডকার ডেভেলপমেন্ট পরিবেশ (`docker-compose.yml`)

বিনা খরচে এবং কোনো আসল ইমেইল না পাঠিয়ে নিজের কম্পিউটারে ইমেইল ভিউ ও টেস্টিং করার জন্য Mailhog সহ ডকার সেটআপ:

```
version: '3.8'

services:
  # ১. পোস্টগ্রেস ডেটাবেজ
  postgres:
    image: postgres:16-alpine
    container_name: zengfy-postgres
    environment:
      POSTGRES_DB: zengfy_core
      POSTGRES_USER: zengfy_user
      POSTGRES_PASSWORD: secret_password
    ports:
      - "5432:5432"
    volumes:
      - pgdata:/var/lib/postgresql/data

  # ২. রেডিস (রেট লিমিট ও ক্যাশিং)
  redis:
    image: redis:7-alpine
    container_name: zengfy-redis
    ports:
      - "6379:6379"

  # ৩. মেইলহগ (ফ্রি লোকাল SMTP সার্ভার)
  # এটি কোনো আসল মেইল পাঠায় না, ব্রাউজারে http://localhost:8025 এ সুন্দর ইনবক্স ভিউ দেয়!
  mailhog:
    image: mailhog/mailhog:latest
    container_name: zengfy-mailhog
    ports:
      - "1025:1025" # SMTP পোর্ট
      - "8025:8025" # ব্রাউজার ওয়েব UI পোর্ট

volumes:
  pgdata:
```

---

৩. ইনসিডেন্ট রেসপন্স ও ট্রাবলশুটিং রানবুক (DevOps SRE Guide)

সমস্যা ১: ইউজারের কাছে ভেরিফিকেশন ইমেইল পৌঁছাচ্ছে না

1. **চেক ১:** Redis কিউ-তে কোনো জব জমে আছে কি না দেখুন: `redis-cli llen bull:emailQueue:wait`
2. **চেক ২:** ডোমেইনের SPF ও DKIM ভ্যালিডেশন চেক করুন: `dig txt zengfy.top +short`
3. **চেক ৩:** ইমেইল লগ দেখতে মেলারের কনটেইনার লগ চালান: `docker logs zengfy-mailhog`

সমস্যা ২: রেজিস্ট্রেশনে বোট অ্যাটাক বা হঠাৎ স্পাইক

1. Cloudflare WAF-এ গিয়ে `/api/v1/auth/register` রুটের জন্য "Under Attack Mode" চালু করুন।
2. Redis স্লাইডিং উইন্ডো লিমিট সাময়িকভাবে ৫ থেকে নামিয়ে ২-এ সেট করুন।

⬅️ *পূর্ববর্তী মডিউল* | 📖 *মূল সূচিপত্র (README)*